

Pic 1

```

23 ⊞ module mac_u (
24     input          i_clk,
25     input          i_rst_n,
26     input  [7:0]   i_a,    // unsigned
27     input  [7:0]   i_b,    // unsigned
28     input  [7:0]   i_c,    // unsigned
29     output reg [7:0] o_y,
30     output reg      o_cout
31 );
32
33 // stage 1: full precision multiply
34 reg [15:0] mult;
35
36 ⊞ always @(posedge i_clk or negedge i_rst_n) begin
37 ⊞     if (!i_rst_n)
38         mult <= 16'd0;
39     else
40 ⊞         mult <= i_a * i_b;
41 ⊞ end
42
43 // stage 2: scale ONCE, then accumulate
44 reg [8:0] sum; // 9-bit to catch overflow
45
46 ⊞ always @(posedge i_clk or negedge i_rst_n) begin
47 ⊞     if (!i_rst_n) begin
48         o_y <= 8'd0;
49         o_cout <= 1'b0;
50 ⊞     end else begin
51         // scale multiplication result ONCE
52         sum = (mult >> 8) + i_c;
53
54         o_y <= sum[7:0];
55         o_cout <= sum[8];
56 ⊞     end
57 ⊞ end
58
59 ⊞ endmodule

```

Pic 2

```

23 module mac_u (
24     input          i_clk,
25     input          i_rst_n,
26     input signed   [7:0] i_a,
27     input signed   [7:0] i_b,
28     input signed   [7:0] i_c,
29     output reg signed [7:0] o_y,
30     output reg signed      o_cout
31 );
32
33 // stage 1: full precision multiply
34 reg signed [15:0] mult;
35
36 always @(posedge i_clk or negedge i_rst_n) begin
37     if (!i_rst_n)
38         mult <= 16'd0;
39     else
40         mult <= i_a * i_b;
41 end
42
43 // stage 2: scale ONCE, then accumulate
44 reg signed [8:0] sum;
45
46 always @(posedge i_clk or negedge i_rst_n) begin
47     if (!i_rst_n) begin
48         o_y <= 8'd0;
49         o_cout <= 1'b0;
50     end else begin
51         // scale multiplication result ONCE
52         sum <= (mult >>> 7) + i_c;
53
54         if (sum[8] != sum[7]) begin
55             o_y <= sum[8] ? 8'b10000000 : 8'b01111111; // saturated
56         end else begin
57             o_y <= sum[7:0]; // no overflow, normal truncation
58         end
59         o_cout <= (sum[8] != sum[7]); // indicate truncation occurred
60     end
61 end
62
63 endmodule

```

mac_u.v

i_a, i_b, i_c Q8.7 signed

i_a * i_b Q8.7 * Q8.7 = ~~Q15.7~~ Q16.14 $\Rightarrow$ mult

i_a * i_b + i_c ~~Q15.7 + Q8.7 = Q16.7~~ Q16.14 + Q8.7 =

Pic1 • Inputs i_a, i_b, i_c are all 8 bits, 7 bits are fractional

Since i_a, i_b, i_c are signed lines 26, 27, 28 changes

26 input signed [7:0] i_a;

27 input signed [7:0] i_b;

28 input signed [7:0] i_c;

And since the inputs are signed, so is "mult"

34 reg signed [15:0] mult;

"mult" is $(X_a X_b) / 2^{14}$

Lines 36 - 41 stay the same, "mult" is 16 bits and the product of i_a and i_b

Assume line 44 is correct at the moment (it is not correct but we'll do it later). Line 48 and 49 are unsure, leave them for now. Now, line 52.

'(mult >> 8)' means $((X_a X_b) / 2^{14}) \times 2^8 = (X_a X_b) / 2^6$, format Q?.6

i_c is format Q8.7. Cannot add. Shift 7 places instead.

52 sum = (mult >>> 7) + i_c;

mult is Q⁷.7, i-c is Q⁸.7

$$X_a, X_b \in [-128, 127]$$

$$(-128) \cdot (-128) = 16384$$

$$X_a X_b \in [-16384, 16384) \quad 15 \text{ bit magnitude} + 1 \text{ sign bit}$$

mult after sift

$$(X_a X_b) / 2^7 \in [-128, 128)$$

i-c

$$X_c / 2^7 \in [-1, 1)$$

range of sum without truncation

$$\text{sum}_{\text{without trunc}} = ((X_a X_b) / 2^7) + (X_c / 2^7)$$

$$\text{range} : [-128 - 1, 128 + 1) = [-129, 129)$$

$$8 \text{ magnitude bits} + 1 \text{ sign bit} \quad Q^{9.7}$$

Going back to line 44, the width is 9 bit, just so happens to match our answer. Cool. But we still need to change the line since it's unsigned

44 reg signed [8:0] sum;

Now lines 54 and 55. We can't just truncate now that there's a signed bit. We gotta use saturation.

$$o_y = \begin{cases} 127 & \text{overflow and pos} \\ -128 & \text{overflow and neg} \\ \text{lower 8 bits} & \text{otherwise} \end{cases}$$

So use if-else to fix this issue.

sum[8] is sign bit, sum[7] is next MSB.

If sign stay the same after truncating sum[8], no overflow.

Example:

• sum = 001010000 (+40)

sum[8] = 0, sum[7] = 0 no overflow

• sum = 010000000 (+128)

sum[8] = 0, sum[7] = 1 overflow

• sum = 10111111 (-129)

sum[8] = 1, sum[7] = 0 overflow

So for line 54 and 55, we do saturation.

54 if (sum[8] != sum[7]) begin

o_y <= sum[8] ? 8'b10000000 : 8'b01111111; // saturate

end else begin

o_y <= sum[7:0]; // no overflow, normal truncation

end

8'b10000000 is -128, max neg

8'b01111111 is +127, max pos

55 o_cont <= (sum[8] != sum[7]); // indicate saturation occurred

Now going back to line 29 and 30, we got the range right, but need to indicate they are signed.

```
29  output reg signed [7:0] o_y;
30  output reg signed      o_cout;
```

Pic 2 Ok, now the range and sign problems should be fixed. Time to fix the cycle problem.

Current:

	mult	sum
cycle 1	0	$0 + i_c 1$
cycle 2	mult 1	$\text{mult } 1 + i_c 2$
cycle 3	mult 2	$\text{mult } 2 + i_c 3$

Should be:

	mult	sum
cycle 1	mult 1	$\text{mult } 1 + i_c 1$
cycle 2	mult 2	$\text{mult } 2 + i_c 2$
cycle 3	mult 3	$\text{mult } 3 + i_c 3$

Must align the pipeline stages. Since "mult" is delayed by 1 cycle, delay i_c by 1 cycle as well. So add a register for i_c in stage 1 and update stage 2.

For stage 1:

after 34 reg signed [7:0] i_c_d;

after 38 i_c_d $\leftarrow$ 8'd0;

after 40 i_c_d $\leftarrow$ i_c;

For stage 2:

52 sum $\leftarrow$ (mult $\ggg$ 7) + i_c_d;